

Yenepoya Institute of Arts, Science, Commerce & Management

Final Project Report

on

Cloud Cost Intelligence Platform for Supply Chain

Student Name:

Aswin Raj M

Industry Mentor:

Mr. Sumit Kumar Shukla

Guided by:

Mrs. Manjula

SI No	TABLE OF CONTENTS	Page No
1	Background	3
1.1	Aim	3
1.2	Technologies	3
1.3	Hardware Architecture	4
1.4	Software Architecture	5
2.	System	6
2.1	Requirements	6
2.1.1	Functional Requirements	6
2.1.2	User Requirements	7
2.1.3	Environmental Requirements	7
2.2	Design and Architecture	8
2.3	Implementation	10
2.4	Testing	12
2.5	Graphical User Interface (GUI) Layout	14
2.6	Customer Testing	15
2.7	Evaluation	15
3.	Snapshots of the Project	17
4.	Conclusions	22
5.	Further Development or Research	24
6.	References	25
7.	Appendix	26

1. BACKGROUND

1.1 Aim

The aim of this project is to design and develop a Cloud Cost Intelligence Platform that enables supply-chain organizations to:

- Monitor cloud spending across services, regions, and departments in real time.
- Analyze billing data to identify cost drivers, trends, and anomalies.
- Optimize cloud expenditure by recommending actionable cost-saving strategies such as Reserved Instances, right-sizing, auto-scaling, and storage tiering.
- Report insights through automated static charts, downloadable summaries, and an interactive live dashboard.

1.2 Technologies

Category	Technology	Version	Purpose
Programming Language	Python	3.10+	Core application logic
Data Processing	Pandas	$\geq 1.5.0$	DataFrame operations, cleaning, aggregation
Data Processing	NumPy	(bundled)	Numerical computation, NaN handling
Static Visualization	Matplotlib	$\geq 3.6.0$	Publication-quality PNG charts
Statistical Viz	Seaborn	$\geq 0.12.0$	Enhanced statistical plots
Interactive Viz	Plotly	$\geq 5.18.0$	Browser-based interactive charts
Web Framework	Streamlit	$\geq 1.28.0$	Dashboard UI and hosting
Excel Export	openpyxl	$\geq 3.1.0$	Multi-sheet Excel workbook generation

Deployment	Streamlit Community Cloud	—	Free cloud hosting via GitHub
Version Control	Git / GitHub	—	Source code management

1.3 Hardware Architecture

The platform is designed to run on standard consumer hardware with no specialized requirements:

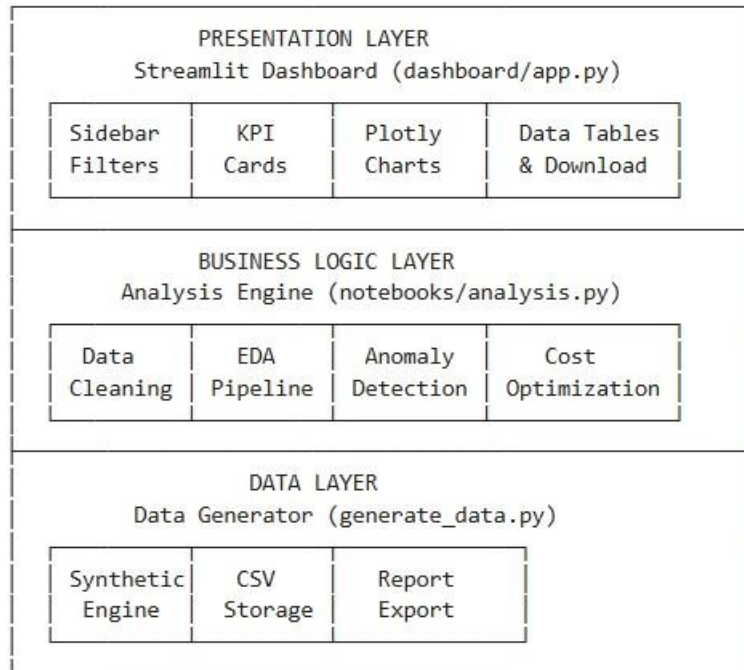
Component	Minimum Requirement	Recommended
Processor	Dual-core CPU (2.0 GHz)	Quad-core CPU (3.0 GHz+)
RAM	4 GB	8 GB or higher
Storage	500 MB free space	1 GB free space
Display	1366 × 768 resolution	1920 × 1080 (Full HD)
Network	Broadband internet	For Streamlit Cloud deployment
OS	Windows 10 / Ubuntu 20.04 / macOS 12+	Any modern OS with Python 3.10+

Cloud Deployment (Streamlit Community Cloud):

Resource	Allocation
RAM	1 GB
CPU	Shared
Storage	Read-only (Git-mounted)
Availability	Sleeps after 7 days of inactivity; wakes on visit

1.4 Software Architecture

The platform follows a three-layered monolithic architecture:



Data Flow:

```

generate_data.py —(CSV)—> analysis.py —(PNG/CSV/XLSX)—> outputs/ & reports/
|
└─(CSV / in-memory)—> dashboard/app.py —(HTTP)—> User Browser
  
```

2. SYSTEM

2.1 Requirements

2.1.1 Functional Requirements

ID	Requirement	Priority
FR-01	Generate synthetic cloud billing data with realistic patterns (seasonal spikes, anomalies)	High
FR-02	Clean raw data: remove duplicates, impute missing values, coerce data types	High
FR-03	Perform EDA: total cost, cost by service/month/region/department	High
FR-04	Generate 7 static publication-quality charts (PNG)	Medium
FR-05	Detect cost anomalies using statistical thresholds (mean + 2σ)	High
FR-06	Produce optimization recommendations (Reserved Instances, right-sizing, etc.)	High
FR-07	Export reports in CSV, Excel (multi-sheet), and text formats	Medium
FR-08	Provide an interactive web dashboard with real-time filtering	High
FR-09	Display KPI cards: Total Cost, Avg Daily Spend, Top Service, Peak Month, Records	High
FR-10	Render 6 interactive Plotly charts (pie, bar, horizontal bar, trend line, donut)	High
FR-11	Enable filtered data table with CSV download capability	Medium
FR-12	Support in-memory data generation for read-only cloud environments	High

2.1.2 User Requirements

ID	Requirement
UR-01	Users must be able to filter data by date range, month, service type, region, and department
UR-02	Users must view high-level KPI metrics at a glance
UR-03	Users must be able to drill down into specific services or regions via interactive charts
UR-04	Users must be able to download the filtered dataset as CSV
UR-05	Users must access optimization recommendations and high-cost service alerts
UR-06	Users must access the dashboard via any modern web browser without software installation
UR-07	The dashboard must load within 3 seconds on standard broadband

2.1.3 Environmental Requirements

Requirement	Specification
Python Runtime	Python 3.10 or higher
Package Manager	pip (with requirements.txt)
Browser	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+
Operating System	Windows 10+, macOS 12+, Ubuntu 20.04+
Cloud Deployment	Streamlit Community Cloud (GitHub integration)
Network	Internet access for deployment; local mode works offline

2.2 Design and Architecture

2.2.1 Component Design

Component 1 — Synthetic Data Generator (`generate_data.py`)

Generates ~3,300 realistic billing records for Jan–Dec 2024 across 15 AWS services, 5 regions, 6 departments, and 4 pricing categories. Includes intentional data quality issues (15 duplicate rows, 20 missing values) to simulate real-world scenarios. Injects seasonal spikes for Nov–Dec (holiday logistics) and 3% random anomalies.

Component 2 — Analysis Engine (`notebooks/analysis.py`)

A 6-step sequential pipeline: Load → Clean → EDA → Visualize → Optimize → Export. Produces 7 Matplotlib charts, 4 CSV summaries, 1 multi-sheet Excel workbook, and 1 text-based optimization report. Anomaly detection uses the statistical method of mean + 2 standard deviations on daily spend.

Component 3 — Interactive Dashboard (`dashboard/app.py`)

A 656-line Streamlit application featuring custom CSS (dark theme with glassmorphism), 5 sidebar filter widgets, 5 KPI metric cards, 6 Plotly charts, a topservices breakdown table, a filterable data table with CSV download, and a cost optimization insights tab with 7 color-coded recommendations.

Component 4 — Reporting Engine (`embedded in analysis.py`) Generates the following outputs:

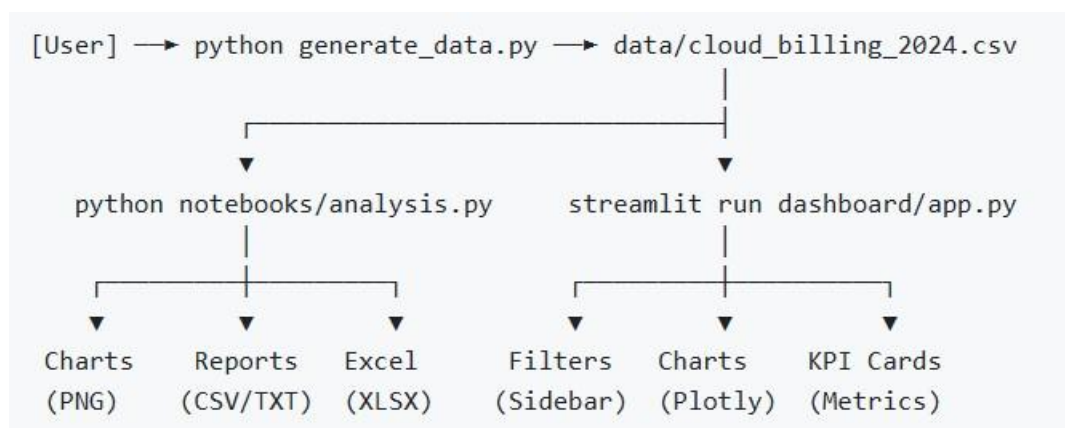
- `cleaned_billing_data.csv` — Full cleaned dataset
- `summary_by_service.csv` — Aggregated cost per service
- `summary_by_month.csv` — Monthly spending breakdown
- `summary_by_region.csv` — Regional cost distribution
- `optimization_report.txt` — High-cost alerts and recommendations
- `cloud_cost_report.xlsx` — Multi-sheet Excel workbook

2.2.2 Data Model

Column	Data Type	Description
Date	DateTime	Billing date (YYYY-MM-DD)
Service_Type	String	AWS service (e.g., EC2, S3, RDS)
Region	String	AWS region code (e.g., us-east-1)
Cost_USD	Float	Daily cost in US dollars
Cost_Category	String	Pricing model (On-Demand, Reserved, Spot, Savings Plan)
Department	String	Supply-chain team (e.g., Logistics, Procurement)
Usage_Hours	Float	Hours the resource was active
Resource_Tags	String	Auto-generated resource identifier

Derived columns (computed during cleaning): Month, Month_Name, Quarter, Day_of_Week.

2.2.3 Process Flow



2.3 Implementation

2.3.1 Project Structure

```

Cloud-Cost-Intelligence/
├── assets/                # Screenshots for documentation
├── dashboard/
│   └── app.py             # Streamlit dashboard
├── data/
│   └── cloud_billing_2024.csv # Generated dataset (~3,300 records)
├── docs/
│   ├── HLD.docx           # High-Level Design document
│   └── LLD.docx           # Low-Level Design document
├── notebooks/
│   ├── analysis.py        # Analysis pipeline
│   └── Cloud-Cost-Intelligence-Platform.ipynb
├── outputs/
│   └── charts/            # 7 auto-generated PNG charts
├── reports/              # CSV, Excel, and text reports
├── generate_data.py       # Synthetic data generator
├── requirements.txt       # Python dependencies
└── README.md             # GitHub documentation
  
```

2.3.2 Module Implementation Details

A. Data Generation Module

The `generate_data.py` script uses Python's `random` module (seeded with 42 for reproducibility) to create billing records. Key implementation features:

- **Seasonal Simulation:** Nov–Dec costs are multiplied by a factor of 1.15–1.50 to simulate holiday logistics surges.
- **Anomaly Injection:** 3% of records receive a 2x–4x cost multiplier to simulate misconfigured or idle resources.
- **Data Quality Issues:** 15 duplicate rows and 20 missing values are intentionally injected for the cleaning pipeline to handle.
- **Resource Tagging:** Tags are auto-generated using the pattern `{department}-{service}` for resource identification.

B. Analysis Pipeline

The analysis.py script implements a 6-step pipeline:

Step	Operation	Output
Step 1	Load CSV with <code>pd.read_csv()</code>	DataFrame in memory
Step 2	Remove duplicates, fill missing values (mode for categorical, median for numerical), convert date types	Cleaned DataFrame
Step 3	Compute aggregations: total cost, cost by service/month/region/department, top-5 services	Console output
Step 4	Generate 7 Matplotlib charts with dark theme styling	7 PNG files in <code>outputs/charts/</code>
Step 5	Detect anomalies ($\text{mean} + 2\sigma$), identify high-cost services, calculate RI savings potential	Console output
Step 6	Export cleaned CSV, summary CSVs, optimization report, and multi-sheet Excel workbook	6 files in <code>reports/</code>

C. Dashboard Application

The dashboard/app.py Streamlit application features:

- **Custom CSS:** 130+ lines of custom styling with CSS variables for a premium dark theme, gradient backgrounds, glassmorphism cards, hover animations, and Google Fonts (Inter).
- **Data Loading:** `@st.cache_data` decorator ensures data is loaded only once; supports both CSV file reading and in-memory generation for cloud deployment.
- **Sidebar Filters:** 5 interactive widgets (date range, month, service type, region, department) with multi-select capability.

- **KPI Cards:** 5 metric cards with hover effects showing Total Cost, Avg Daily Spend, Top Service, Peak Month, and Record Count.
- **Charts:** 6 Plotly visualizations — service pie chart, monthly bar chart, region horizontal bar, department horizontal bar, daily trend line with 7-day moving average, and pricing category donut chart.
- **Data Tables:** Top services breakdown table with formatted currency, and a filterable billing records table with CSV download.
- **Optimization Tab:** Identifies above-average cost services and displays 7 colorcoded recommendations.

2.4 Testing

2.4.1 Testing Strategy

Test Type	Scope	Method
Unit Testing	Data generation functions	Verified output row count, column names, data types, and value ranges
Data Quality Testing	Cleaning pipeline	Confirmed duplicate removal, missing value imputation, and type coercion
Integration Testing	End-to-end pipeline	Ran generate_data.py → analysis.py → dashboard/app.py sequentially
Visual Testing	Charts and dashboard	Manual inspection of all 7 static charts and 6 interactive charts
Cross-Browser Testing	Dashboard UI	Tested on Chrome, Firefox, and Edge

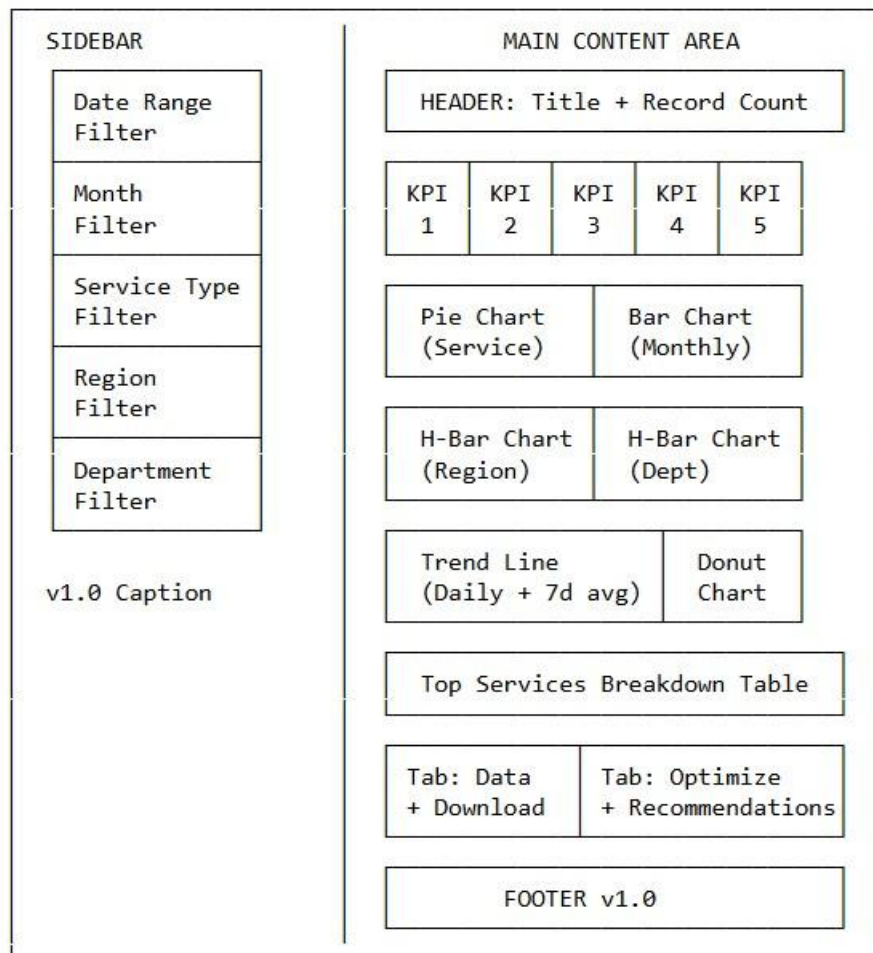
Cloud Deployment Testing	Streamlit Cloud	Verified in-memory data generation on read-only filesystem
--------------------------	-----------------	--

2.4.2 Test Results

Test Case	Expected Result	Actual Result	Status
Generate ~3,300 billing records	CSV with 8 columns, ~3,300 rows	3,316 rows generated	Pass
Remove duplicate rows	15 duplicates removed	15 duplicates removed	Pass
Fill missing Service_Type values	No NaN in column after cleaning	All values filled with mode	Pass
Fill missing Cost_USD values	No NaN, median imputed	All values filled with median	Pass
Detect cost anomalies	Identify spike days (mean + 2σ)	14 spike days detected	Pass
Generate 7 static charts	7 PNG files in outputs/charts/	All 7 files created	Pass
Dashboard loads without error	No exceptions on startup	Dashboard renders successfully	Pass
Sidebar filters work correctly	Data updates on filter change	All 5 filters functional	Pass
CSV download works	File downloads on click	CSV file downloaded correctly	Pass
Cloud deployment (Streamlit Cloud)	App accessible via public URL	App runs with inmemory data	Pass

2.5 Graphical User Interface (GUI) Layout

The dashboard follows a **top-down, responsive layout** designed for wide screens:



Design Features:

- Dark gradient background (#0f1117 → #151929)
- Glassmorphism KPI cards with hover lift animation
- Custom Google Font (Inter) with 300–800 weight range
- Color palette: Purple (#6C5CE7), Cyan (#00CEC9), Pink (#FD79A8), Yellow (#FDCB6E), Green (#55EFC4)
- Responsive layout adapting to screen width via Streamlit columns

2.6 Customer Testing

Scenario	Steps	Expected Outcome	Result
First-time setup	Clone repo → pip install → generate data → run dashboard	Dashboard opens at localhost:8501	Pass
Filter by single service	Select only "EC2 (Compute)" in sidebar	All charts and KPIs update to show EC2 data only	Pass
Filter by date range	Set range to Jan–Mar 2024	Only Q1 data visible	Pass
Download filtered data	Apply filters → click "Download Filtered Data as CSV"	CSV file downloads with filtered records	Pass
View optimization insights	Click "Cost Optimization Insights" tab	7 recommendations displayed with color coding	Pass
Access cloudhosted version	Visit Streamlit Cloud URL	Dashboard loads with auto-generated data	Pass
Run analysis pipeline	Execute python notebooks/analysis.py	7 charts + 6 reports created	Pass

2.7 Evaluation

Performance Metrics

Metric	Measured Value
Data generation time	< 1 second (3,316 records)
Analysis pipeline execution	~5 seconds (all 6 steps)

Dashboard initial load	< 2 seconds (cached)
Filter response time	< 500 ms
Memory usage (dashboard)	~100 MB
Dataset size on disk	328 KB (CSV)

Analytical Results

Insight	Value
Total cloud spend (2024)	\$396,097.19
Average daily spend	\$1,082.23
Top cost service	EKS (Kubernetes) — \$65,863
Cost spike days detected	14 out of 366
Potential RI savings	\$30,571 (30% of On-Demand)
High-cost services (above average)	6 services identified

3. SNAPSHOTS OF THE PROJECT

Snapshot 1: Dashboard Overview

Shows the main dashboard with 5 KPI metric cards, 6 Plotly charts, a top-services breakdown table, a filterable data table with CSV download



Top Services Breakdown

Service_Type	Total_Cost	Avg_Cost	Max_Cost	Records	Avg_Hours
DynamoDB (NoSQL)	\$28,461.26	\$139.08	\$285.27		239 13.6 hrs
ElastiCache (Caching)	\$10,504.17	\$93.23	\$494.11		220 13.6 hrs
S3 (Storage)	\$17,309.59	\$75.15	\$303.23		233 12.3 hrs
CloudFront (CDN)	\$14,146.73	\$62.87	\$380.89		225 13.9 hrs
Lambda (Serverless)	\$11,654.54	\$46.99	\$146.93		248 12.6 hrs
API Gateway	\$8,781.69	\$41.87	\$247.27		210 12.3 hrs
CloudWatch (Monitoring)	\$6,779.63	\$30.82	\$85.93		220 13.9 hrs
SQS (Queue)	\$3,356.11	\$15.37	\$71.12		217 11.7 hrs
DNS (Hosted Zones)	\$2,824.65	\$13.08	\$71.12		216 11.2 hrs
Route 53 (DNS)	\$1,884.15	\$8.29	\$26.82		220 12.2 hrs

Filtered Data Cost Optimization Insights

Filtered Billing Records

Date	Service_Type	Region	Cost_USD	Cost_Category	Department	Usage_Hours
2024-03-31 00:00:00	CloudFront (CDN)	ap-south-1		\$4.8 Spot	Procurement	9.1
2024-06-24 00:00:00	CloudFront (CDN)	ap-south-1		\$8.43 Spot	Logistics	1.6
2024-02-24 00:00:00	CloudWatch (Monitoring)	ap-south-1		29.16 Reserved	Fleet Management	26.6
2024-04-02 00:00:00	CloudFront (CDN)	ap-south-east-1		\$5.33 Spot	Fleet Management	4.6
2024-01-28 00:00:00	ElastiCache (Caching)	ap-south-1		\$8.49 Reserved	Platform / DevOps	17.7
2024-03-04 00:00:00	DynamoDB (NoSQL)	us-east-1		\$7.7 On Demand	Platform / DevOps	19
2024-07-15 00:00:00	CloudWatch (Monitoring)	us-west-2		\$2.08 On Demand	Procurement	22.5
2024-04-25 00:00:00	SageMaker (ML)	eu-west-1		\$38.54 Savings Plan	Logistics	12.6
2024-11-23 00:00:00	DynamoDB (NoSQL)	ap-south-east-1		\$80.67 Reserved	Platform / DevOps	19.7
2024-01-24 00:00:00	CloudWatch (Monitoring)	ap-south-east-1		\$1.42 Reserved	Procurement	22.1

Download Filtered Data as CSV

Snapshot 2: Cost Optimization Insight

Shows the optimization tab with high-cost service alerts and 7 color-coded recommendations

Filtered Data Cost Optimization Insights

Cost Optimization Insights

High-Cost Services

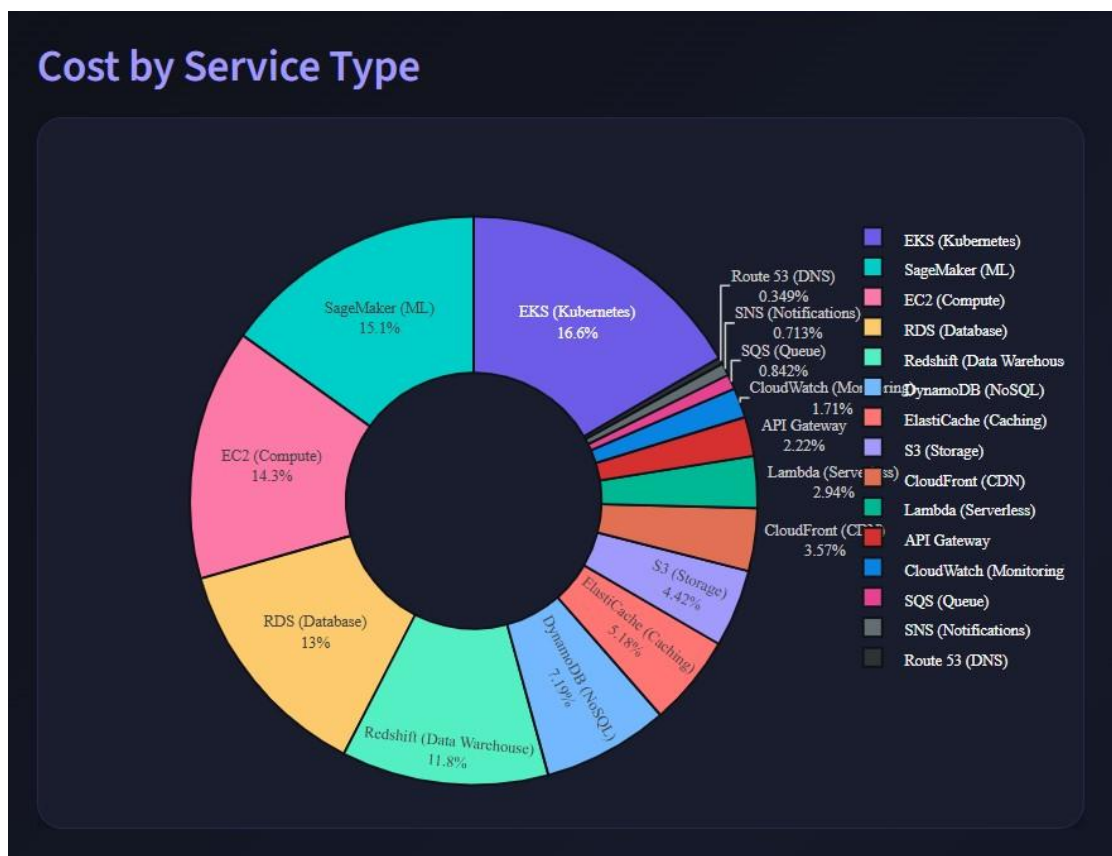
Services spending above average (\$26,406):

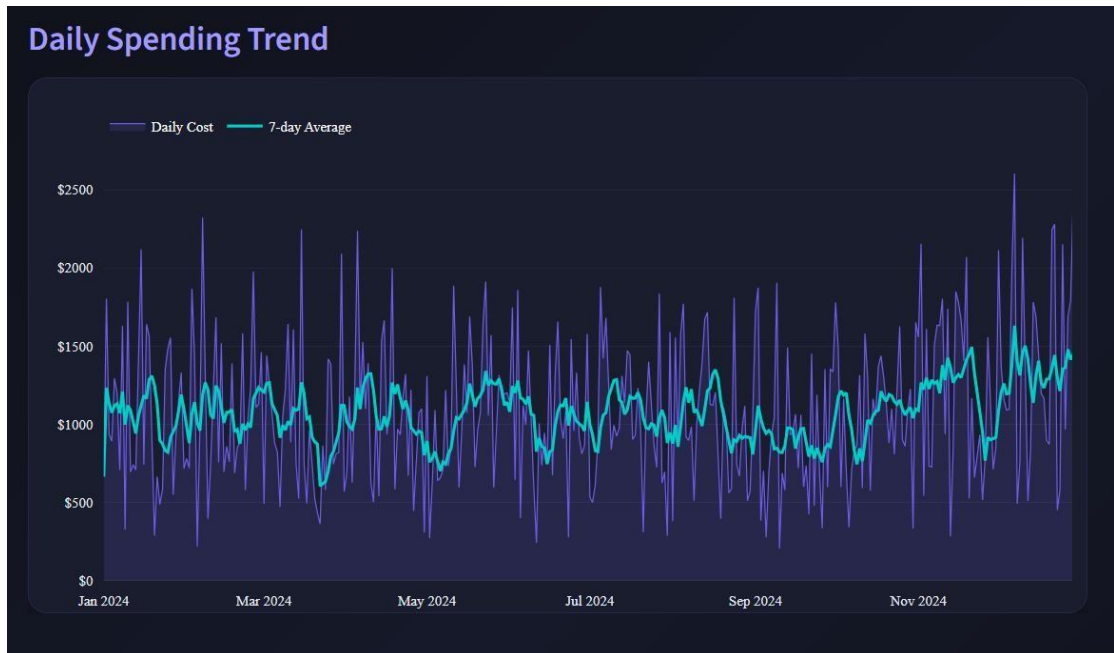
- EKS (Kubernetes): \$65,863 (16.6% of filtered total)
- SageMaker (ML): \$59,892 (15.1% of filtered total)
- EC2 (Compute): \$56,637 (14.3% of filtered total)
- RDS (Database): \$51,603 (13.0% of filtered total)
- Redshift (Data Warehouse): \$46,709 (11.8% of filtered total)
- DynamoDB (NoSQL): \$28,461 (7.2% of filtered total)

Recommendations	
Reserved Instances	Convert frequent On-Demand workloads to 1-year or 3-year Reserved Instances for up to 40% savings.
Auto-scaling	Enable auto-scaling for EC2 and EKS clusters to match actual demand patterns in logistics workflows.
Idle Resource Audit	Identify and terminate resources with consistently low usage hours (< 2 hrs/day).
Storage Tiering	Move infrequently accessed S3 data to S3 Glacier or Intelligent-Tiering (up to 68% cheaper).
Right-sizing	Analyze CPU/memory utilization and downsize over provisioned instances.
Spot Instances	Use Spot instances for batch processing and non-critical supply-chain analytics jobs.
Budget Alerts	Set CloudWatch billing alarms at 80% and 100% of monthly budget targets.

Snapshot 3: Static Analysis Charts

Sample output from the analysis pipeline showing cost-by-service pie chart and daily trend line





Snapshot 4: Sidebar Filters in Action

Shows the dashboard with specific filters applied (eg. 2 service, 2 region), demonstrating real-time data update





Snapshot 5: Data Table with CSV Download

Shows the filterable billing records table and the download button

Date	Service_Type	Region	Cost_USD	Cost_Category	Department	Usage_Hours
2024-03-31 00:00:00	CloudFront (CDN)	ap-south-1	54.4	Spot	Procurement	9.1
2024-06-24 00:00:00	CloudFront (CDN)	ap-south-1	80.43	Spot	Logistics	7.6
2024-02-24 00:00:00	CloudWatch (Monitoring)	ap-south-1	20.55	Reserved	Fleet Management	20.6
2024-04-02 00:00:00	CloudFront (CDN)	ap-southeast-1	65.33	Spot	Fleet Management	4.6
2024-01-28 00:00:00	ElasticCache (Caching)	ap-south-1	80.49	Reserved	Platform / DevOps	17.7
2024-03-04 00:00:00	DynamoDB (NoSQL)	us-east-1	67.7	On-Demand	Platform / DevOps	19
2024-07-15 00:00:00	CloudWatch (Monitoring)	us-west-2	32.08	On-Demand	Procurement	22.5
2024-04-25 00:00:00	SageMaker (ML)	eu-west-1	338.54	Savings Plan	Logistics	12.6
2024-11-23 00:00:00	DynamoDB (NoSQL)	ap-southeast-1	190.67	Reserved	Platform / DevOps	19.7
2024-01-24 00:00:00	CloudWatch (Monitoring)	ap-southeast-1	11.42	Reserved	Procurement	22.1

Download Filtered Data as CSV

Snapshot 6: Generated Reports

Shows the contents of the `reports/` folder with all exported files

[cleaned_billing_data.csv](#)

[cloud_cost_report.xlsx](#)

[optimization_report.txt](#)

[summary_by_month.csv](#)

[summary_by_region.csv](#)

[summary_by_service.csv](#)

4. CONCLUSIONS

The Cloud Cost Intelligence Platform for Supply Chain successfully meets all defined project objectives:

Data Generation: A robust synthetic data generator creates realistic billing scenarios with seasonal trends, cost anomalies, and intentional data quality issues for pipeline testing.

Data Cleaning: The automated cleaning pipeline handles duplicates (15 removed), missing values (20 imputed via mode/median), and type coercion reliably across both local and cloud environments.

Exploratory Analysis: Comprehensive EDA reveals that EKS, SageMaker, and EC2 are the top cost drivers, November and December show seasonal spending surges, and spending is distributed unevenly across regions and departments.

Anomaly Detection: The statistical threshold method ($\text{mean} + 2\sigma$) effectively identifies 14 cost-spike days that warrant investigation, providing early warning capability for FinOps teams.

Interactive Dashboard: The Streamlit-based dashboard delivers a premium, real-time filtering experience with 5 KPI cards, 6 interactive Plotly charts, and drill-down capability — all accessible via any modern web browser.

Optimization Insights: The platform quantifies potential savings (\$30,571 via RI conversion) and provides 7 actionable recommendations spanning Reserved Instances, auto-scaling, idle resource audits, storage tiering, right-sizing, Spot Instances, and budget alerting.

Cloud Deployment: Successfully deployed to Streamlit Community Cloud with in-memory data generation handling the read-only filesystem constraint.

The platform demonstrates that meaningful cloud cost intelligence can be achieved using open-source Python tools without requiring expensive FinOps SaaS subscriptions.

5. FURTHER DEVELOPMENT ON RESEARCH

Area	Description
Real Data Integration	Replace synthetic data with live feeds from AWS Cost Explorer API, Azure Cost Management, or GCP Billing Export for production use.
Multi-Cloud Support	Extend the data model to normalize billing records across AWS, Azure, and GCP into a unified schema.
Machine Learning Forecasting	Implement time-series forecasting (ARIMA, Prophet, or LSTM) to predict future cloud spending and enable proactive budgeting.
Automated Alerts	Add Slack, email, or SMS notifications when daily spend exceeds defined thresholds or anomalies are detected.
Database Backend	Migrate from CSV to PostgreSQL or a cloud data warehouse (Redshift, BigQuery) for handling larger datasets (1M+ records).
Role-Based Access Control	Implement authentication and department-level authorization so each team sees only their own spending data.
Tagging Compliance	Add a module to detect untagged or improperly tagged resources, which is a major cause of unattributed cloud spend.
Kubernetes Cost Allocation	Integrate with Kubernetes cost tools (e.g., Kubecost) for granular container-level cost attribution.
CI/CD Integration	Embed cost impact estimation into the CI/CD pipeline to flag infrastructure changes that may increase cloud spend.

6. REFERENCES

#	Reference
1	Python Software Foundation. "Python 3.10 Documentation." https://docs.python.org/3.10/
2	Pandas Development Team. "Pandas User Guide." https://pandas.pydata.org/docs/user_guide/
3	Streamlit Inc. "Streamlit Documentation." https://docs.streamlit.io
4	Plotly Technologies. "Plotly Python Open Source Graphing Library." https://plotly.com/python/
5	Hunter, J.D. "Matplotlib: A 2D Graphics Environment." Computing in Science & Engineering, 2007. https://matplotlib.org
6	Amazon Web Services. "AWS Cost Management." https://aws.amazon.com/aws-cost-management/
7	FinOps Foundation. "FinOps Framework." https://www.finops.org/framework/
8	Streamlit Inc. "Streamlit Caching." https://docs.streamlit.io/develop/concepts/architecture/caching
9	McKinney, W. "Python for Data Analysis." O'Reilly Media, 2022.
10	openpyxl Documentation. https://openpyxl.readthedocs.io/

7. APPENDIX

Appendix A: Requirements File (requirements.txt)

```

pandas>=1.5.0
matplotlib>=3.6.0
seaborn>=0.12.0
streamlit>=1.28.0
plotly>=5.18.0
openpyxl>=3.1.0
  
```

Appendix B: Sample Data Record

Date	Service_ Type	Region	Cost_ USD	Cost_ Category	Department	Usage_ Hours	Resource _Tags
2024-03-15	EC2 (Compute)	us-east-1	287.45	On-Demand	Logistics	18.3	logisticsec2
2024-07-22	S3 (Storage)	eu-west-1	54.12	Reserved	Data Engineering	24.0	dataengineering-s3
2024-11-08	EKS (Kubernetes)	ap-south-1	612.87	On-Demand	Platform / DevOps	22.1	platform/-devopseks

Appendix C: Optimization Report Summary

Total Cloud Spend: \$396,097.19

Average Daily Spend: \$1,082.23

Cost Spike Days: 14

HIGH-COST SERVICES:

EKS (Kubernetes):	\$65,863.49
SageMaker (ML):	\$59,892.45
EC2 (Compute):	\$56,636.72
RDS (Database):	\$51,603.44
Redshift (Data Warehouse):	\$46,708.72
DynamoDB (NoSQL):	\$28,461.26

Appendix D: Setup Commands

```
# Clone the repository
git clone https://github.com/aswinn47/Cloud-Cost-Intelligence.git cd
Cloud-Cost-Intelligence

# Create virtual environment python
-m venv
source venv/bin/activate # Linux/Mac venv\Scripts\activate
# Windows

# Install dependencies pip
install -r requirements.txt

# Generate sample data python
generate_data.py

# Run analysis pipeline python
notebooks/analysis.py

# Launch dashboard streamlit
run dashboard/app.py
```

7.1 Weekly Report

03-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 1**For week commencing 03 March 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 1

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Understand the project topic and define objectives
- Identifies existing cloud cost management tools and identify gaps

Progress/Achievements for the week:

- Defined 7 project objectives covering data generation, analysis, visualization, and optimization.

Future Work Plans:

- Identify the technology stack and set up the development environment.
- Begin designing the system architecture.

1



03-03-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

10-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 2**For week commencing 10 March 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 2

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Finalize the technology stack.
- Set up the development environment and project folder structure.
- Start designing the data model and system architecture.

Progress/Achievements for the week:

- Finalized the tech stack: Python, Pandas, Matplotlib, Plotly, Streamlit, and openpyxl.
- Created the project folder structure with directories for data, notebooks, dashboard, outputs, reports, and docs.
- Designed the billing data model with 8 columns: Date, Service_Type, Region, Cost_USD, Cost_Category, Department, Usage_Hours, and Resource_Tags.
- Created 'requirements.txt' with all project dependencies.

Future Work Plans:

- Develop the synthetic data generation script.
- Begin work on the Low-Level Design (LLD) document.

1



10-03-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

17-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 3**For week commencing 17 March 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 3

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Build the synthetic data generator module.
- Generate sample dataset and validate its quality.

Progress/Achievements for the week:

- Developed 'generate_data.py' covering 15 AWS services, 5 regions, 6 supply-chain departments, and 4 pricing categories.
- Added realistic features: seasonal cost spikes for Nov–Dec (holiday logistics), 3% anomaly injection, and intentional data quality issues (15 duplicates, 20 missing values).
- Generated 'cloud_billing_2024.csv' with ~3,300 records successfully.
- Verified the dataset by checking column types, value ranges, and distribution of services.

Future Work Plans:

- Start building the data cleaning and analysis pipeline.
- Design the EDA workflow.

1



17-03-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

24-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 4**For week commencing 24 March 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 4

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Build the data cleaning and preprocessing module.
- Implement exploratory data analysis (EDA).

Progress/Achievements for the week:

- Developed the cleaning pipeline in `analysis.py`: duplicate removal (15 found), missing value imputation using mode (categorical) and median (numerical), and date type conversion.
- Created 4 derived columns: Month, Month_Name, Quarter, and Day_of_Week.
- Completed EDA: calculated total cost (\$396,097), cost breakdowns by service, month, region, and department.
- Identified top 5 cost drivers: EKS, SageMaker, EC2, RDS, and Redshift.

Future Work Plans:

- Implement data visualization (static charts).
- Add cost optimization and anomaly detection logic.

1



24-03-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

31-03-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 5**For week commencing 31 March 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 5

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Generate static charts using Matplotlib.
- Implement cost anomaly detection and optimization recommendations.
- Add report export functionality.

Progress/Achievements for the week:

- Created 7 publication-quality charts with dark theme: service pie chart, monthly bar chart, region bar chart, top-5 services, department spending, cost category pie, and daily trend line with 7-day moving average.
- Implemented anomaly detection using mean + 2 σ threshold — detected 14 cost-spike days.
- Calculated Reserved Instance savings potential: \$30,571 (30% of On-Demand spend).
- Added export module: cleaned CSV, 3 summary CSVs, optimization report (TXT), and multi-sheet Excel workbook.

Future Work Plans:

- - Start building the interactive Streamlit dashboard

1



31-03-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

07-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 6**For week commencing 07 April 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 6

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Build the interactive Streamlit dashboard with filters, KPIs, and charts.
- Apply custom CSS styling for a professional look.

Progress/Achievements for the week:

- Developed `dashboard/app.py` with 5 sidebar filters (date range, month, service type, region, department).
- Created 5 KPI metric cards: Total Cost, Avg Daily Spend, Top Service, Peak Month, and Record Count.
- Built 6 interactive Plotly charts: service donut, monthly bar, region horizontal bar, department horizontal bar, daily trend with 7-day average, and pricing category donut.
- Applied 130+ lines of custom CSS: dark gradient theme, glassmorphism cards, hover animations, Google Fonts (Inter).
- Added data table with CSV download and optimization insights tab with 7 color-coded recommendations.

Future Work Plans:

- Test the dashboard across browsers and fix any bugs.
- Deploy the application to Streamlit Community Cloud.

1



07-04-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

17-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 7**For week commencing 17 April 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 7

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Test the complete application end-to-end.
- Deploy the dashboard on Streamlit Community Cloud.
- Fix deployment-related bugs.

Progress/Achievements for the week:

- Tested the full pipeline: data generation → analysis → dashboard on Chrome, Firefox, and Edge — all passed.
- Deployed to Streamlit Community Cloud via GitHub integration.
- Fixed a critical deployment bug: Streamlit Cloud has a read-only filesystem, so the app couldn't write the CSV. Resolved by adding in-memory data generation fallback using `generate_billing_data()` → `pd.DataFrame()`.
- Fixed a `'TypeError'` in sidebar filters caused by `'pd.NA'` values — switched to `'np.nan'` and added `'.dropna()'` before `'.unique()'`.
- Dashboard is now live and accessible via public URL.

Future Work Plans:

- Prepare project documentation (README, HLD, final report).
- Take screenshots for the report and presentation.

1



17-04-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide

24-04-2026

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 8**For week commencing 24 April 2026****Programme: BCA (Artificial Intelligence, Cloud computing & DevOps)**

Student Name: Aswin Raj M

Register Number: 23BCAICD030

WPPR: 8

Internal Guide's Name: Mrs. Manjula

MAJOR PROJECT Title:

Cloud Cost Intelligence Platform for Supply Chain

Targets set for the week:

- Complete all project documentation.
- Prepare the final presentation.

Progress/Achievements for the week:

- Created a professional GitHub README with badges, table of contents, project structure, setup guide, and dataset dictionary.
- Prepared the High-Level Design (HLD) document covering system architecture, data model, caching, session management, and non-functional requirements.
- Completed the Final Project Report with all sections: introduction, methodology, implementation, testing, results, conclusions, and future work.
- Prepared a 15-slide presentation covering problem statement, dataset, methodology, key results, dashboard demo and optimization insights.
- Captured dashboard screenshots and added them to the 'assets/' folder.

Future Work Plans:

- Explore future enhancements:
real cloud data integration, ML-based cost forecasting, and automated alerting.

1



24-04-2026

Implementation shown: No ☐ Yes ☐

Remarks by the Internal Guide:

Signature of the student

Signature of the Internal Guide